

7. KEYWORDS AS ARGUMENT LABELS:

Up till now, Swift did not allow the use of identifiers such as `func`, `let`, `public` and `class` as keyword names. Even though this behaviour wouldn't seem odd to a seasoned programmer, newbies are likely to cause problems in their code. Even so, one was still allowed to use keywords as argument labels by putting them in ```. To make code more readable and identifiers more intuitive, Swift developers have now allowed for any keyword to be used as an argument label, with the exception of `inout`, `var`, and `let`. If you have code that used keywords in backticks, you'll get an Xcode Fix-it to remove them. Pretty neat, if you ask me.

8. AUTOMATICALLY SYNTHESIZED HEADERS:

Since Objective C is a superset of the C programming language, the ability to view header files, which only exposed the methods and parameters without any of their implementations, was present. Not so much with Swift. One would have to scroll through 1000s of lines of codes just to be able to learn how to call a particular function. With the release of Swift 2, Xcode is also now able to produce synthesized files – it scans through your entire codebase and produces a condensed virtual header that provides a summary of your functions.

9. THE “GUARD” KEYWORD:

Since a lot of the changes made by Apple developers seem to focus on syntactic intuitiveness and reducing the learning curve for newer programmers, the introduction of the guard keyword makes a lot more sense. In summary, the usage of guard is very similar to `if`, but with the use of guard, code readability shoots up dramatically and the ultimate intention becomes much clearer. An example would be to check for non-zero length input when a Submit button is tapped.

With a proposed release date of late 2016, many more changes are lined up for Swift 3. One thing is clear though: Apple wants you to program in Swift, and it's pulling out all stops to make sure its brainchild is adopted worldwide, faster and sooner than any other language. And we programmers couldn't possibly ask for more, could we? **d**